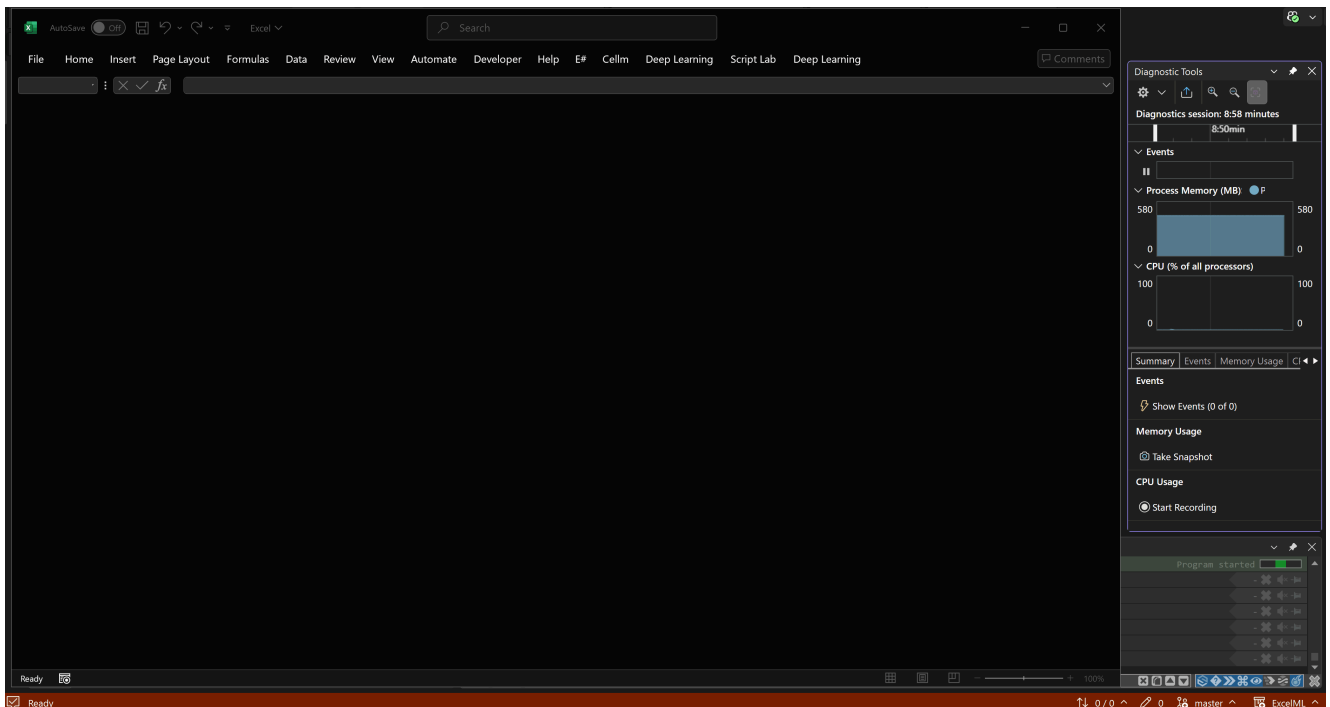


ExcelML

Deep learning and modern machine learning, natively inside Excel — for learning, experimentation, and explainability.

Vision



Live XOR training with push-based updates inside Excel

ExcelML is an experimental platform that brings *real* machine learning and deep learning workflows directly into Microsoft Excel — without Python, Jupyter, or external services. We are developing an Excel-based environment to familiarize our clients and business partners with basic ML concepts in the most approachable way we can think of. For the moment we're limiting ourselves to core ML concepts, but we plan to eventually extend it to LLM's as well.

The key idea is simple but ambitious:

If someone understands Excel formulas, ranges, and tables, they should be able to learn and experiment with modern ML concepts using those same mental models.

ExcelML is **not** intended to compete with PyTorch, TensorFlow, or scikit-learn as production ML frameworks. Instead, it is designed as:

- a **learning-first environment** for ML and deep learning
- a **transparent and inspectable** alternative to black-box notebooks
- a **bridge** for Excel-native users into modern ML concepts

Why Excel?

Excel already provides many of the things ML learners struggle to reason about:

- Structured data (tables, named ranges)

- Deterministic recalculation
- Immediate visual feedback
- Built-in charting
- A low-friction UI familiar to millions

What Excel lacks is:

- Tensor operations
- Differentiation
- Optimizers
- Model state

ExcelML fills that gap — *without* turning Excel into a scripting environment.

Core Design Principles

1. Excel remains Excel

- No VBA macros
- No COM automation during training
- No Python runtimes
- No hidden background services

All ML functionality is exposed as **pure Excel functions (UDFs)**.

2. Explicit, user-controlled computation

ExcelML avoids accidental recomputation by design.

Training is always explicit:

```
=DL.TRAIN(model_id, X, y, opts, trigger)
```

- Recalculation alone does **not** retrain models
- A user-controlled **trigger token** governs training
- Training is deterministic and inspectable

This mirrors good ML hygiene while fitting Excel's recalc model.

3. Small models, fully inspectable

ExcelML intentionally focuses on:

- Small neural networks
- Simple datasets (XOR, regression, classification)
- CPU-only execution

This allows:

- Inspecting weights in cells
 - Visualizing loss curves
 - Exploring activations layer-by-layer
 - Understanding *why* a model learns
-

4. Real ML frameworks under the hood

Although the interface is Excel-native, ExcelML uses **real ML infrastructure**:

- **TorchSharp** (the .NET bindings for PyTorch)
- Automatic differentiation
- Modern optimizers (Adam, SGD, etc.)

This ensures concepts learned in ExcelML transfer directly to industry tools.

Architecture Overview

```
Excel
├─ Worksheets (ranges, tables, charts)
├─ UDFs (DL.MODEL_CREATE, DL.TRAIN, ...)
├─
Excel-DNA Add-in (.xll)
├─ Async-safe UDF execution
├─ Trigger-aware recalculation
├─ In-process model registry
├─
Managed .NET Layer
├─ Model lifecycle & state
├─ Tensor conversion (Excel ≠ Torch)
├─ Logging & diagnostics
├─
TorchSharp
├─ PyTorch-compatible tensors
├─ Autograd
├─ Optimizers
├─ Native libtorch (CPU)
```

All computation happens **in-process** with Excel, minimizing marshalling overhead and maximizing transparency.

Solution Structure

- **ExcelML (solution)**
 - Vision, documentation, and experiments
- **ExcelDIPlayground (project)**
 - Excel-DNA add-in
 - TorchSharp integration
 - UDFs and Ribbon UI
 - Practical experimentation ground

The solution-level documentation (this file) describes *why* the project exists. The project-level docs describe *how* it works.

Intended Audience

ExcelML is aimed at:

- Excel power users curious about ML
- Data analysts transitioning toward ML
- Educators teaching ML fundamentals
- Engineers exploring explainability

It is *not* aimed at:

- Large-scale production ML
 - GPU-heavy workloads
 - Automated model deployment
-

What ExcelML Is *Not*

To set expectations clearly:

- Not AutoML
- Not a replacement for Python notebooks
- Not optimized for large datasets
- Not a production inference engine

ExcelML values **clarity over speed, learning over scale**.

Roadmap (Conceptual)

Phase 1 — Foundations (current)

- TorchSharp integration
- Explicit training triggers
- Loss history visualization
- XOR / small MLPs

Phase 2 — Inspectability

- `DL.PREDICT`
- `DL.WEIGHTS`
- `DL.ACTIVATIONS`
- Layer-by-layer visualization

Phase 3 — Teaching workflows

- Guided Excel workbooks
- Built-in examples
- Visualization templates

Phase 4 — Persistence & sharing

- Model serialization
 - Workbook portability
 - Reproducible experiments
-

Why This Matters

Most ML learning tools optimize for *convenience*, not *understanding*.

ExcelML takes the opposite approach:

If you can see it, you can understand it.

By anchoring ML concepts in a familiar, visual, deterministic environment, ExcelML lowers the barrier to entry — while keeping the concepts honest.

Status

This project is **experimental**, exploratory, and intentionally opinionated.

It exists to answer a single question:

What would machine learning look like if it were designed for understanding first?

ExcelML is our attempt to find out.

See Also

- `AGENTS.md` — hard-won technical lessons and constraints
 - `ExcelD1Playground/README.md` — project-level usage and setup
-